

Session 11. Solving a final exam

We need to develop an application for marking students' exams.

On one side, we have the **description of the exam**. The exam contains several **questions** of different **types**: Choice, Fill-in-the-gap and Value.

- A **multiple choice** question has four possible answers (a, b, c, d) but only one is right.
- A **gap filling** question will leave a gap to be filled by a word.
- In a **value** question the student has to calculate a numeric (double) result.

Each question also contains a **score** and a **question number** that comes from its position in the exam.

On the other side, we have the **students' answers**.

The purpose of this application will be to compare them with correct answers and mark the exam.

1 Files containing information

questions.txt describes the exam. Each line refers to one question and follows the syntax

```
<type of question>\t<weight>\t<correct answer>
```

studentsExams.gz, a text file compressed in gz format, contains students' answers. Each line contains **all answers for a single student**. That is, students will answer each and every question (you must assume that no question is left unanswered).

The format for each line is:

```
<studentID>\t<answer to question 1>\t<answer to question 2>\t<ans... 3>\t<...>
```

Sample files:

questions.txt	studentsExams.gz
choice 1.0 a	20215 a b stuff thread 13.5 80
choice 1.0 b	20214 b c thing computer 14.5 0
gap 0.5 computer	20213 c d matter computer 12.5 10
...	...
value 1.5 100.0	...

2 Mark the exams

The marking procedure consists of several steps.

1. Loading info from files
2. Marking student's exam generating students' marks.
3. Saving the marks into a file sorted by id or by mark.

3 Loading info from files

To mark the exams, the app must load the description of the exam from *questions.txt* and the students' answers from *studentsExams.gz*.

Loading information from *questions.txt* will generate a list of objects of type **Question** (see figure 1) while loading information from *studentsExams.gz* will generate a list of objects of type **StudentExam** (see figure 2).

LOADING

questions.txt

```
choice 1.0 a
choice 1.0 b
gap 0.5 stuff
gap 0.5 computer
value 1.5 12.5
value 1.5 100.0
gap 1.0 polymorphism
value 1.0 256.0
choice 0.5 c
gap 1.5 abstract
```

reading

```
"choice 1.0 a"
"choice 1.0 b"
"gap 0.5 stuff"
"gap 0.5 computer"
"value 1.5 12.5"
"value 1.5 100.0"
"gap 1.0 polymorphism"
"value 1.0 256.0"
"choice 0.5 c"
"gap 1.5 abstract"
```

parsing

```
1 Choice->{1.0, "a"}
2 Choice->{1.0, "b"}
3 Gap->{0.5, "stuff"}
4 Gap->{0.5, "computer"}
5 Value->{1.5, 12.5}
6 Value->{1.5, 100.0}
7 Gap->{1.0, "polymorphism"}
8 Value->{1.0, 256.0}
9 Choice->{0.5, "c"}
10 Gap->{1.5, "abstract"}
```

Figure 1: Reading and parsing questions

students_answers.txt.gz

```
20209 c b cosa computer 10.5 100 inheritance 256.0 a abstract
20210 a b stuff processor 11.5 120 inheritance 128.0 a abstract
20211 b a other process 12.5 110 inheritance 256.0 a blank
20215 a b stuff thread 13.5 80 inheritance 0.0 a n-a
20214 b c thing computer 14.5 0 inheritance 512.0 a abstract
20213 c d cosa computer 12.5 10 inheritance 1021.0 a abstract
```

LOADING

reading

```
"20209 c b cosa computer 10.5 100 inheritance 256.0 a abstract"
"20210 a b stuff processor 11.5 120 inheritance 128.0 a abstract"
"20211 b a other process 12.5 110 inheritance 256.0 a blank"
"20215 a b stuff thread 13.5 80 inheritance 0.0 a n-a"
"20214 b c thing computer 14.5 0 inheritance 512.0 a abstract"
"20213 c d cosa computer 12.5 10 inheritance 1021.0 a abstract"
```

parsing

```
"20209", {"c", "b", "cosa", "computer", "10.5", "100", "inheritance", "256.0", "a", "abstract"}
"20210", {"a", "b", "stuff", "processor", "11.5", "120", "inheritance", "128.0", "a", "abstract"}
"20211", {"b", "a", "other", "process", "12.5", "110", "inheritance", "256.0", "a", "blank"}
"20215", {"a", "b", "stuff", "thread", "13.5", "80", "inheritance", "0.0", "a", "n-a"}
"20214", {"b", "c", "thing", "computer", "14.5", "0", "inheritance", "512.0", "a", "abstract"}
"20213", {"c", "d", "cosa", "computer", "12.5", "10", "inheritance", "1021.0", "a", "abstract"}
```

Figure 2: Reading and parsing answers

4 Marking student's exam generating students' marks

```
"20209", {"c", "b", "cosa", "computer", "10.5", "100", "inheritance", "256.0", "a", "abstract"} 20209, 7.5
"20210", {"a", "b", "stuff", "processor", "11.5", "120", "inheritance", "128.0", "a", "abstract"} 20210, 8.0
"20211", {"b", "a", "other", "process", "12.5", "110", "inheritance", "256.0", "a", "blank"} marking 20211, 4.5
"20215", {"a", "b", "stuff", "thread", "13.5", "80", "inheritance", "0.0", "a", "n-a"} 20215, 6.0
"20214", {"b", "c", "thing", "computer", "14.5", "0", "inheritance", "512.0", "a", "abstract"} 20214, 7.3
"20213", {"c", "d", "cosa", "computer", "12.5", "10", "inheritance", "1021.0", "a", "abstract"} 20213, 9.8
```

Figure 3: Marking all students

1. Class **ExamMarker** contains a method **mark()** to mark all questions in all students' exams.

This method will iterate students' exams in the list marking all them one by one and generating an instance of class **StudentMark** for each one.

```

void mark() {
    for(StudentExam se: answers) {
        this.marks.add(new StudentMark ( se.getId(), se.mark(questions)));
    }
}

```

2. Class `StudentExam` is responsible for calculating its own mark by adding the points of every individual question.

```

class StudentExam {
    private String id;
    private List<String> answers;

    public double mark(List<Question> questions) {
        double res = 0.0;
        for (int i = 0; i < answers.size(); i++) {
            Question q = questions.get(i);
            String answer = answers.get(i);
            res += q.mark(answer);
        }
        return res;
    }
}

```

3. Finally, class `Question` takes the responsibility of generating each question's mark.

```

public abstract class Question {
    public abstract double mark(String answer);
}

```

Depending on the type of question, the points scored are calculated differently:

- **Multiple Choice:** If the answer matches the right option, he receives the score for that question. Otherwise, he gets -20% of the score.
- **Value:** If the answer is in the range $[\text{correct answer}-0.1 - \text{correct answer}+0.1]$, the question is awarded the total score. Otherwise, 0.
- **Fill-in-the-gap:** If the answer is correct, the total score is received. Otherwise, 0.

5 Writing marks in an output file

The final marks will be written into a text file (`marks.txt`), one student per line, by ascending student id. The mark will be written with one decimal place.

The format of each line will be:

```
student_id-->student grade
```

6 Sort

The list of marks must be sorted in two different ways: by natural order (**student identification**) or by mark and, for the same mark, by identification.

Write the necessary methods to sort by both criteria so both can be displayed on the console.

To sort a list you should use the sort method of the `java.util.Collections` class.

Then, try to use our own lists. You will need to implement the sort algorithm.

7 Validation

Consider just the following and ignore any other possible source of error:

- All arguments must be non-null, including lists.
- All Strings must also be non-blank.
- In the constructor of `Question`, `weight` and `questionNumber` must be greater than zero.
- In the constructor of `StudentExam`, the identifier must be length five and contain just digit chars.

8 How to handle errors in the application

8.1 User errors

Parse errors Assume that `studentsExams.gz` is **error-free**. Consider the following errors in `questions.txt`

- wrong number format, that is, where a number is expected, the string at the input contains characters that can not be parsed into a valid number,
- unknown question types, other than choice, gap o value,
- more or less fields than expected, that is, each line describing a question must contain exactly the following fields: the type of question, the weight and the right answer, and
- weight must be greater than zero.

Your application must not stop execution. Just write a proper message on a log file, ignore the wrong line and continue. The message to be written in the log file must be:

```
INVALID LINE <line-number>: <message describing the error found at the line>
```

File not found If some input file does not exist, then the exception will be `FileNotFoundException` with a message `<filename> does not exist`.

Repeated exams When there is more than one answer for the same student identifier, we consider the input file was corrupted and so the information loaded from it is invalid. Throw a custom exception `ExamMarkerException` with an explaining message `REPEATED EXAM STUDENT ID <ident>`.

Both, file not found and repeated exams, will cause the application to finish the execution and print the message

```
USER ERROR: <message>. Solve the problem and try again.
```

8.2 Programming errors

All programming errors and `IOException` except `FileNotFoundException` will stop the execution gracefully, will write a message in `System.err` and will log the stack trace in the log file. The message printed will be:

```
PROGRAMMING ERROR. <description of the error>  
Contact your system admin.
```

9 Write tests

Write JUnit test for all the following methods:

- `QuestionParser::parse(...)`
- `ExamMarker::loadStudentExams(...)` but **do not test parsing question**
- `Serializer::serialize(...)`
- method `mark(...)` in all the Question hierarchy.

10 Important remarks

- Don't round any value. Consider using `Math.floor` to get the significant decimals before formatting the double. For one significant decimal:

```
double v = Math.floor(mark * 10) / 10.0;
```